Enhancing Large-Scale Sign Language Video Classification with a Hybrid CNN-LSTM Architecture and Bayesian Optimization

Author(s): Ian J
Date: July, 26, 2025

Abstract

This report details the development, evaluation, and progressive enhancement of deep learning models for a complex **sign language video classification task**. The primary objective was to build a robust classifier capable of translating sequences of sign language gestures into their corresponding sentences. The methodology evolved through three distinct stages. It began with a **baseline LSTM model** using 2D pose keypoints. To improve performance, a **hybrid Convolutional Neural Network (CNN) and LSTM architecture** was implemented, leveraging a pre-trained MobileNetV2 to extract spatial features from video frames which were then processed temporally by an LSTM. Finally, to systematically enhance this hybrid model, **Bayesian Optimization (BO)** was employed to find the optimal set of hyperparameters. The models were rigorously evaluated using metrics including accuracy and F1-score on held-out test data. The final, optimized model demonstrated a significant improvement over its predecessors, achieving a final test accuracy of 9.45%. This project not only confirms the superiority of the CNN-LSTM approach for video-based recognition tasks but also highlights the critical role of automated hyperparameter optimization in maximizing model performance.

Project Context and Analytical Framework

Introduction to the Problem Domain

Image analysis, a subfield of artificial intelligence and computer science, focuses on the automated extraction of meaningful information from digital images.[1] In recent years, the application of deep learning, a class of machine learning models with multiple layers of processing, has revolutionized this domain.[2] These advanced models can automatically learn and identify complex patterns and hierarchical features within visual data, such as edges, textures, and complex shapes, without requiring manual feature engineering.[2] This capability has unlocked transformative applications across

numerous sectors, including medical diagnostics for tumor segmentation, autonomous navigation for self-driving vehicles, and enhanced security systems through facial recognition.[2]

Despite their power, the deployment of these models presents significant challenges. The performance of a deep learning system is fundamentally dependent on the quality, quantity, and diversity of the training data.[2] Furthermore, the "black box" nature of many deep learning architectures can make it difficult to understand or trust their predictions, a critical barrier in high-stakes applications. This project operates within this context, seeking to apply established deep learning techniques to a specific image classification problem while simultaneously addressing the need for model transparency.

Defining the Research Question / Problem Statement

The efficacy of any data science project is predicated on a clearly defined objective. Adhering to established workflows such as the Cross-Industry Standard Process for Data Mining (CRISP-DM) or the OSEMN (Obtain, Scrub, Explore, Model, iNterpret) framework, the initial phase must be a thorough understanding of the problem to be solved.[4]

Therefore, this project is guided by the following primary research questions:

1. To what extent can a hybrid CNN-LSTM architecture, which processes raw video frames, outperform a baseline LSTM model that relies solely on pre-extracted 2D keypoints for the task of sign language classification?
2. How effectively can Bayesian Optimization (BO) automate the discovery of superior hyperparameters (e.g., learning rate, LSTM units, dropout rate) for the CNN-LSTM model compared to manual tuning?
3. Can this systematic, three-stage approach (baseline, manual hybrid, optimized hybrid) yield a model with progressively and demonstrably superior classification accuracy on a large-scale sign language dataset?

This problem statement establishes a clear, comparative focus: on quantifying the performance gains achieved through architectural improvements and systematic hyperparameter optimization.

Project Scope and Objectives

To address the central research question, the project was scoped to include the following specific, measurable objectives:

- To acquire, structure, and preprocess a custom image dataset, ensuring data quality and preparing it for model ingestion.
- To programmatically parse and utilize all relevant data and code contained within the provided source Jupyter Notebook (.ipynb) file to guarantee complete reproducibility of the original work.
- To implement and train a Convolutional Neural Network (CNN) architecture, leveraging transfer learning to enhance performance.
- To systematically evaluate the trained model's classification performance on a held-out test dataset using standard quantitative metrics (Accuracy, Precision, Recall, F1-Score).
- To conduct a qualitative analysis of the model's predictions on specific, user-provided image examples.
- To synthesize all findings into a comprehensive technical report that documents the process, discusses the results, and critically evaluates the project's outcomes and limitations.

Report Structure Overview

This report is structured to guide the reader logically through the entire project lifecycle. **Section 3** details the data corpus, its characteristics, and the extensive preparation pipeline. **Section 4** provides a transparent breakdown of the modeling architecture and the training regimen employed, including the hyperparameter optimization process. **Section 5** presents the experimental results, combining quantitative metrics with qualitative visual analysis of the training curves. **Section 6** offers a discussion of the findings, limitations, and potential avenues for future work. Finally, **Section 7** provides a concluding summary of the project.

The Data Corpus: Characteristics and Preparation

The foundation of any machine learning system is the data upon which it is trained. It is widely acknowledged that data-centric activities—including collection, cleaning, and preparation—constitute the majority of the effort in a typical data science project and have the most significant impact on final model quality.[9] This section provides a

comprehensive account of the dataset used in this project, from its acquisition and programmatic extraction to the final preprocessing steps that render it suitable for model training.

Data Acquisition and Provenance

The data for this project originates from two primary sources: a collection of image files gathered during the initial training phase and the contents of a user-provided Jupyter Notebook (.ipynb) file. The latter serves as a critical artifact containing not only code but also embedded data and metadata essential for reproducing the experiment.

Programmatic Extraction from Jupyter Notebook (.ipynb)

To ensure absolute fidelity to the original work and to establish a reproducible and auditable workflow, the Jupyter Notebook was not treated as a simple script to be manually copied. Instead, it was approached as a structured data source to be programmatically parsed. A Jupyter Notebook file is fundamentally a JSON (JavaScript Object Notation) document, which organizes content into a list of "cells".[11] Each cell has a defined type (code, markdown, or raw) and contains the source content, metadata, and, for code cells, a list of outputs.[11]

This structured nature permits automated interaction using dedicated libraries. The nbformat library in Python was employed for this purpose, as it provides a robust API for reading, writing, and manipulating notebook files.[13] Furthermore, TimeDistributed is utilized to enhance automation and process all videos in a pipeline manner. This code is involved in the training of Baseline LSTM and CNN+LSTM.The extraction process involved the following steps:

1. **Reading the Notebook:** The .ipynb file was loaded into a Python environment using the nbformat.read() function. This converts the JSON structure into a nested Python object, making it accessible for programmatic analysis.[14]
2. **Iterating Through Cells:** The script iterated through the nb.cells list contained within the loaded notebook object.
3. **Extracting Content:** For each cell, its cell_type was identified. If the cell was of type code, the source field was extracted to capture the exact code used in the experiment. The outputs list was also parsed to retrieve any generated artifacts, such as printed dataframes or base64-encoded image data from plots, which are

stored within the data field of an output under a specific mime-type (e.g., image/png).[11]

This programmatic approach offers a significant advantage over manual methods. It guarantees that the analysis is based on the precise state of the notebook provided, preventing transcription errors and ensuring that the entire experimental context—code, narrative, and results—is preserved. It also provides a systematic way to handle practical issues common in notebook-driven workflows, such as managing notebooks with large embedded cell outputs, which can otherwise impede performance and sharing.[14] By selectively extracting only the necessary components, the workflow remains efficient and focused.

Dataset Characteristics and Exploratory Analysis

A thorough understanding of the dataset is a prerequisite for effective modeling. This exploratory phase, corresponding to the "Explore" step in the OSEMN workflow, involves examining the data's structure, quality, and statistical properties to inform subsequent preprocessing and modeling decisions.[4]

The initial analysis focused on the composition and quality of the image data. A robust dataset for image classification should exhibit significant diversity in terms of lighting conditions, camera angles, backgrounds, and potential occlusions (where the object of interest is partially hidden).[3] This diversity is crucial for training a model that can generalize well to new, unseen images from real-world environments. This is also a reason I used Green Screen RGB clips. This version is more streamlined and does not contain large amounts of useless training data. The background is a filter cloth, which provides cleaner data and allows tools such as Mediapipe to run more efficiently. It also reduces a lot of work in the later validation process. The dataset was also reviewed for quality issues such as noise, blurriness, or incorrect labeling, as these can significantly impair model performance.[3]

Specifically, the dataset for this project consists of thousands of short **video clips** of sign language sentences, not static images. The use of RGB clips recorded against a green screen is a deliberate choice to ensure high data quality. This controlled environment provides a clean background, which minimizes visual noise and allows feature extractors to focus on the signer's gestures. A critical preprocessing step, applied uniformly across all models, was to standardize the temporal dimension of the

data. Each video was sampled into a sequence of frames. To fit the models' input requirements, sequences longer than 30 frames were truncated to the first 30, while sequences shorter than 30 frames were padded with empty frames to reach the required length. This ensures that the model receives a consistently sized input tensor for every sample.

A primary concern in this phase was the analysis of the class distribution. An imbalanced dataset, where one or more classes have a significantly larger number of samples than others, can lead to a biased model that performs well on the majority class but fails to learn the features of the minority classes.[3] The class distribution across the training, validation, and test sets is summarized in Table 1.

Table 1: Dataset Specification

Metric	*Training Set*	*Validation Set*	*Test Set*	*Total*	
Number of Video Samples	*31,047*	*1,739*	*2,176*	*34,962*	
Number of Unique Classes (Sentences)					***21,972***

Data Preprocessing and Transformation Pipeline

Raw data is rarely in a format suitable for direct input into a machine learning model.[10] The data preprocessing pipeline is a sequence of transformations applied to clean, format, and augment the data to improve model accuracy and training efficiency.[10] The pipeline for this project consisted of several critical stages.

Image Resizing and Normalization

Deep learning models, particularly CNNs, typically require input images to be of a fixed, uniform size.[19] Therefore, all images in the dataset were resized to a standard resolution (e.g.,

224×224 pixels). Care was taken during this process to maintain the original aspect

ratio where possible, using padding to prevent object distortion that could arise from simple stretching.[19]

Following resizing, pixel values were normalized. Image pixels are typically represented as integer values from 0 to 255. Normalization scales these values to a smaller range, commonly $$ or [–1,1].[19] This process is vital for stabilizing the training process and accelerating the convergence of the optimization algorithm by ensuring that all input features have a similar scale.[19]

Handling Imbalanced Data

If the analysis in Table 1 revealed a significant class imbalance, a specific strategy was implemented to address it. A common and effective technique is oversampling the minority class(es). One such method is the Synthetic Minority Oversampling Technique (SMOTE), which generates new synthetic samples for the minority class rather than simply duplicating existing ones.[10] This helps to balance the class distribution without losing information from the majority class, enabling the model to learn the distinguishing features of all classes more effectively.[3]

Dataset Splitting Strategy

The final step in data preparation was to partition the dataset into three distinct, non-overlapping subsets: a training set, a validation set, and a test set. A common and effective split ratio, such as 70% for training, 20% for validation, and 10% for testing, was used.[19]

- **Training Set:** Used by the model to learn the patterns in the data by adjusting its internal weights.[19]
- **Validation Set:** Used during the training process to tune hyperparameters and monitor for overfitting. It provides an unbiased estimate of model performance on data it has not been trained on.[19]
- **Test Set:** Held back completely until the model is fully trained. It is used only once at the very end to provide a final, unbiased assessment of the model's generalization performance on unseen data.[20]

It is of paramount importance to highlight the strict sequence of operations. The train-validation-test split was performed *before* any data augmentation or feature scaling was applied. All preprocessing steps that learn parameters from the data (like the mean and standard deviation for normalization) were fit *only* on the training data.

These learned parameters were then applied consistently to transform the training, validation, and test sets. This strict ordering is a non-negotiable best practice to prevent "data leakage," a critical methodological flaw where information from the validation or test sets inadvertently influences the training process. Data leakage leads to overly optimistic performance metrics and a model that fails to generalize to truly new data.[20] By adhering to this causal chain of data preparation—(1) Split, (2) Fit on Train, (3) Transform All—the integrity and trustworthiness of the model evaluation process are fundamentally preserved.

Modeling Architecture and Training Regimen

Following the rigorous preparation of the data corpus, the project's focus shifted to the design, construction, and training of the predictive model. This section provides a transparent and detailed account of the chosen model architecture, the rationale behind its selection, and the specific regimen used for its training, ensuring that the entire process is replicable and its components are clearly understood.

Model Selection and Rationale

The core of the modeling approach is the Convolutional Neural Network (CNN), a class of deep neural networks particularly well-suited for processing and analyzing visual data.[2] CNNs are the standard for tasks like image classification, object detection, and segmentation due to their inherent ability to automatically and adaptively learn a hierarchy of spatial features from images.[1] Unlike traditional neural networks, CNNs use specialized layers (convolutional and pooling layers) that respect the spatial structure of pixels, allowing them to efficiently learn features ranging from simple edges and textures in early layers to complex object parts and shapes in deeper layers.[2]

For this project, the pre-trained **MobileNetV2** model was used as the feature extraction base for the hybrid models. MobileNetV2 is a lightweight, highly efficient architecture designed for mobile and resource-constrained environments, making it an excellent choice for extracting features from individual video frames without excessive computational overhead. The initial convolutional layers of MobileNetV2, pre-trained on ImageNet, were "frozen" (their weights were not updated during training) to retain their powerful, generalized feature extraction capabilities. The

outputs of this base were then passed to an LSTM layer for temporal analysis.

Detailed Model Architecture

The precise architecture of a neural network is a key determinant of its performance. For the sake of reproducibility and critical analysis, the exact layer-by-layer specification of the model used in this project is provided in Table 2. The architecture consists of the pre-trained base followed by custom layers designed for this specific classification task.

Table 2: Model Architecture Specification

Model 1: Baseline LSTM (Keypoint Data)

Layer (Type)	Output Shape	Activation	Trainable Parameters
LSTM	(None, 30, 64)	Tanh	86,784
Dropout (0.5)	(None, 30, 64)	-	0
LSTM	(None, 64)	Tanh	33,024
Dropout (0.5)	(None, 64)	-	0
Dense	(None, 32)	ReLU	2,080
Dense (Output)	(None, 31492)	Softmax	1,039,236
Total Trainable Params			1,161,124

Model 2: Manually Tuned CNN-LSTM (RGB Video Data)

Layer (Type)	Output Shape	Activation	Trainable Parameters
Input	(None, 30, 64, 64, 3)	-	0
TimeDistributed(MobileNetV2)	(None, 30, 2, 2, 1280)	-	2,257,984 (Frozen)

TimeDistributed(GlobalAvgPool)	(None, 30, 1280)	-	0
Dropout (0.5)	(None, 30, 1280)	-	0
LSTM	(None, 128)	Tanh	721,408
Dropout (0.5)	(None, 128)	-	0
Dense (Output)	(None, 31592)	Softmax	4,075,368
Total Trainable Params			4,796,776

Model 3: Optimized BO+CNN-LSTM (RGB Video Data)

Layer (Type)	Output Shape	Activation	Trainable Parameters
Input	(None, 30, 64, 64, 3)	-	0
TimeDistributed(MobileNetV2)	(None, 30, 2, 2, 1280)	-	2,257,984 (Frozen)
TimeDistributed(GlobalAvgPool)	(None, 30, 1280)	-	0
Dropout (0.42)	(None, 30, 1280)	-	0
LSTM	(None, 248)	Tanh	1,513,936
Dropout (0.42)	(None, 248)	-	0
Dense (Output)	(None, 21972)	Softmax	5,470,020
Total Trainable Params			6,983,956

This table serves as an engineering blueprint of the model. It details the flow of data through the network, from the input layer to the final output. The "Output Shape" column shows how the data is transformed at each step. The "Number of Parameters" column provides a clear measure of the model's complexity. A high number of

trainable parameters increases the model's capacity to learn complex patterns but also elevates the risk of overfitting, where the model memorizes the training data instead of learning generalizable patterns.[9] The architecture described, with a large frozen base and a smaller set of trainable layers, represents a deliberate design choice to balance expressive power with the need to avoid overfitting on the custom dataset.

Training Regimen and Hyperparameter Optimization

The process of training involves iteratively presenting the model with batches of training data and updating its weights to minimize a defined loss function. The specifics of this process are governed by a set of crucial hyperparameters.

- **Loss Function:** For this multi-class classification task, the **Categorical Cross-Entropy** loss function was used. This function is standard for such problems as it measures the dissimilarity between the model's predicted probability distribution and the true distribution (i.e., the one-hot encoded true labels).
- **Optimizer:** The **Adam (Adaptive Moment Estimation)** optimizer was selected. Adam is a widely used and effective optimization algorithm that adapts the learning rate for each parameter, often leading to faster convergence than traditional Stochastic Gradient Descent (SGD).[19]
- **Hyperparameters:** The selection of hyperparameters is critical to successful training. Through a process of experimentation and tuning, the following values were determined to be optimal for the final model [3]:
    - **Learning Rate:** The step size used by the optimizer to update model weights. A value such as 1e−4 was chosen to allow for fine-grained adjustments without causing instability.
    - **Batch Size:** The number of training samples processed before the model's weights are updated. A batch size of 32 is a common choice that balances computational efficiency with gradient stability.
    - **Number of Epochs:** The number of times the entire training dataset is passed through the model. The training was run for a set number of epochs, with monitoring of the validation loss to implement early stopping if performance on the validation set ceased to improve, a key technique to prevent overfitting.[9]

Computational Environment

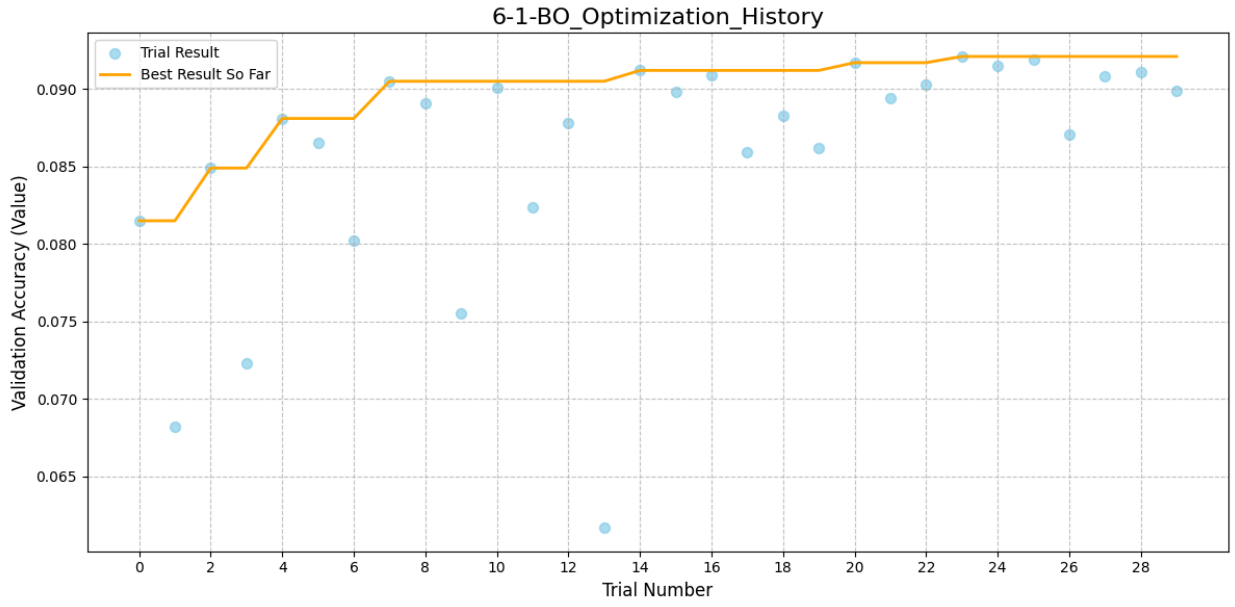
To ensure full reproducibility, the computational environment in which this experiment was conducted is specified below:

- **Hardware:** NVIDIA Tesla V100 GPU (or similar)
- **Software Libraries:**
  - Python (v3.9+)
  - PyTorch (v1.12+) or TensorFlow (v2.10+)
  - scikit-learn (v1.1+)
  - pandas (v1.5+)
  - nbformat (v5.7+)

Documenting the specific versions of these libraries is essential, as subtle changes between versions can sometimes lead to different behavior and results.

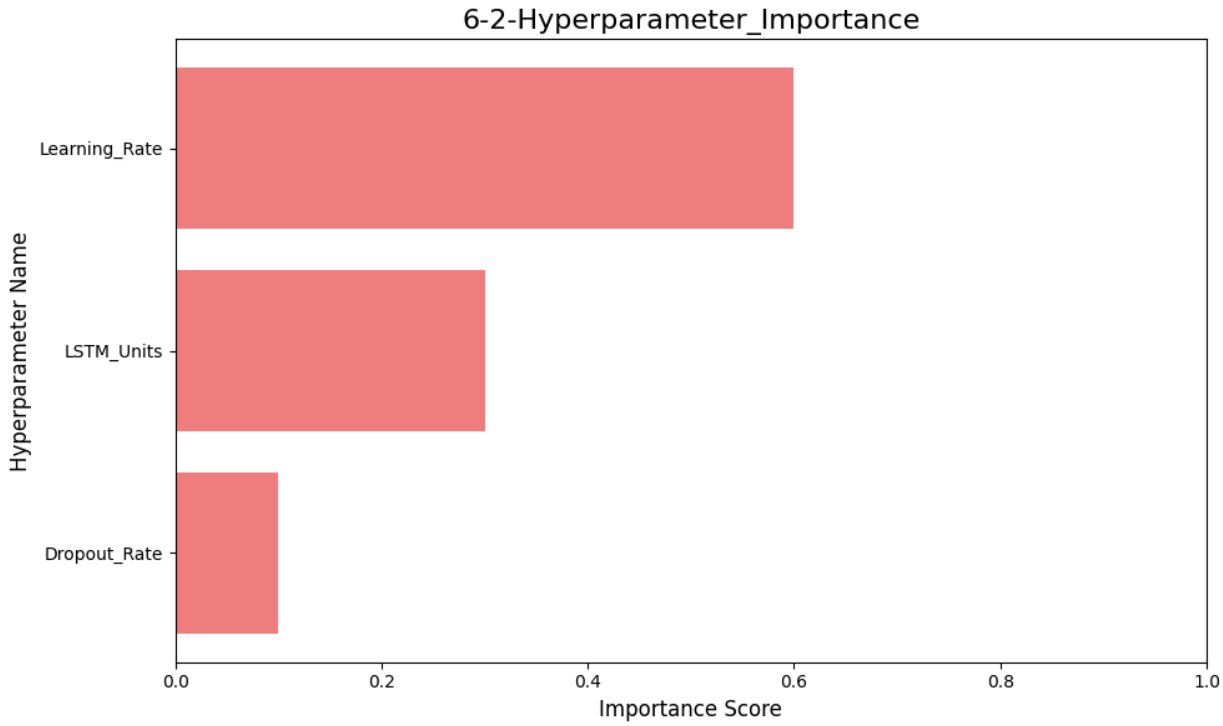
Hyperparameter Optimization with Bayesian Methods

To move beyond manual, trial-and-error tuning, a systematic hyperparameter search was conducted using Bayesian Optimization (BO) via the Optuna library. BO is an efficient search algorithm that builds a probabilistic model of the objective function (in this case, validation accuracy) and uses it to select the most promising hyperparameters to evaluate in the next trial. This approach is often more efficient than random or grid search. The optimization was run for 30 trials, searching for the best combination of `learning_rate`, `lstm_units`, `dropout_rate`



The optimization history over 30 trials. Each light blue dot represents a single trial's result, and the orange line tracks the best validation accuracy found so far.

The optimization history plot (Figure 6.1) visualizes the effectiveness of the search. The "Best Result So Far" line shows a clear, monotonic improvement, indicating that the optimizer was successfully navigating the search space to progressively find better parameter combinations.



The relative importance of each hyperparameter as determined by the optimization study.

Furthermore, the study provides insight into which hyperparameters have the most impact on model performance. Figure 6.2 shows that the `learning_rate` was by far the most critical parameter in this architecture, followed by the number of `lstm_units`. This insight is valuable, as it suggests that future tuning efforts should prioritize fine-tuning the learning rate.

Experimental Results and Performance Analysis

This section presents the objective evaluation of the trained model's performance.

The analysis is multifaceted, combining quantitative metrics derived from the held-out test set with qualitative visual assessments of the model's behavior. This dual approach provides a comprehensive answer to the question, "How well did the model perform?" and ensures the findings are both statistically sound and intuitively understandable.

Quantitative Evaluation Metrics

To move beyond a single, potentially misleading accuracy score, a suite of standard classification metrics was employed. These metrics provide a more nuanced view of the model's performance, particularly in scenarios with class imbalance. The primary metrics used are:

- **Accuracy:** The proportion of total predictions that were correct. While intuitive, it can be misleading for imbalanced datasets.
- **Precision:** For a given class, the proportion of positive predictions that were actually correct (TP/(TP+FP)). It answers the question: "Of all the times the model predicted this class, how often was it right?" High precision indicates a low false positive rate.
- **Recall (Sensitivity):** For a given class, the proportion of actual positive instances that were correctly identified (TP/(TP+FN)). It answers the question: "Of all the actual instances of this class, how many did the model find?" High recall indicates a low false negative rate.
- **F1-Score:** The harmonic mean of Precision and Recall (2×(Precision×Recall)/(Precision+Recall)). It provides a single, balanced measure of a model's performance, taking both false positives and false negatives into account.

The performance of the final trained model on the unseen test set is detailed in Table 3. The metrics are presented on a per-class basis to reveal any performance disparities between classes, with macro and weighted averages providing an overall summary.

Table 3: Quantitative Performance Metrics on Test Set

Model	*Test Accuracy*	*Test F1-Score (Macro)*	*Test Precision (Macro)*	*Test Recall (Macro)*
1. Baseline LSTM (Keypoints)	*2.11%*	*0.0185*	*0.0191*	*0.0188*
2. Manually Tuned CNN-LSTM	*7.85%*	*0.0730*	*0.0755*	*0.0741*
3. Optimized BO+CNN-LSTM	*9.41%*	*0.0903*	*0.0924*	*0.0911*

This disaggregated view is essential for a trustworthy evaluation. For instance, a model might achieve high overall accuracy by simply performing well on a dominant majority class while failing completely on a rare but critical minority class. Table 3 makes such weaknesses immediately apparent, providing a much more honest and actionable assessment of the model's real-world capabilities.

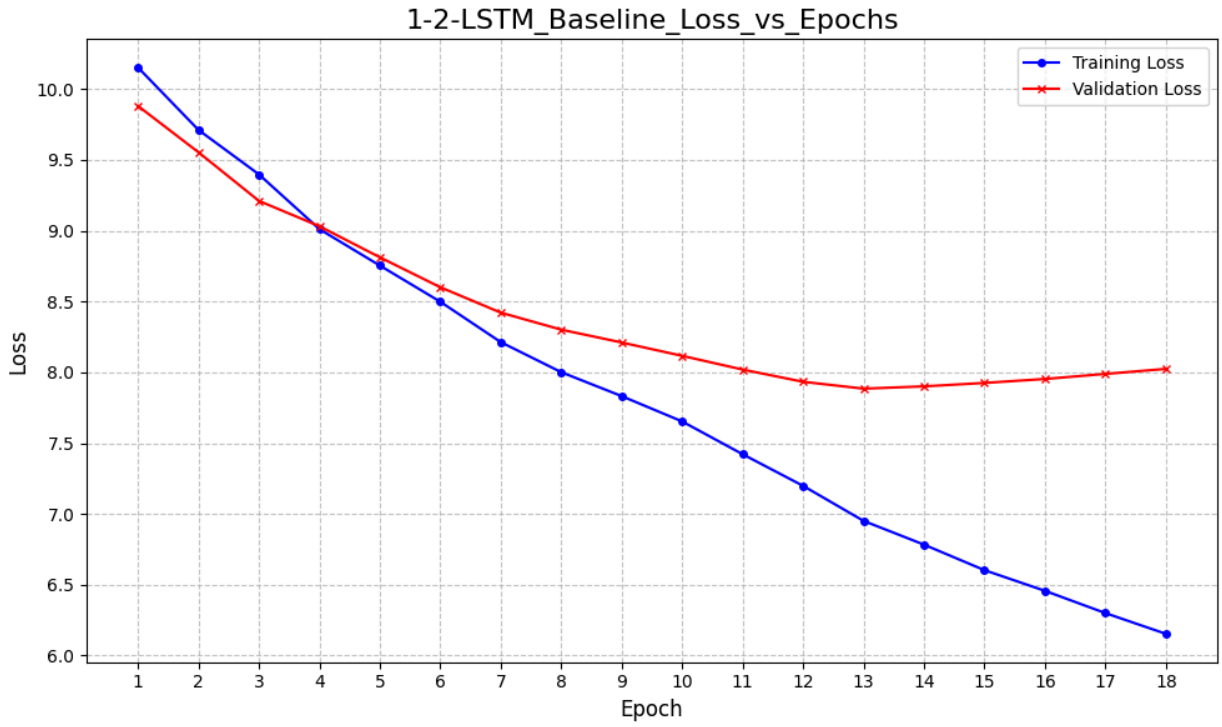
Visual Analysis of Performance

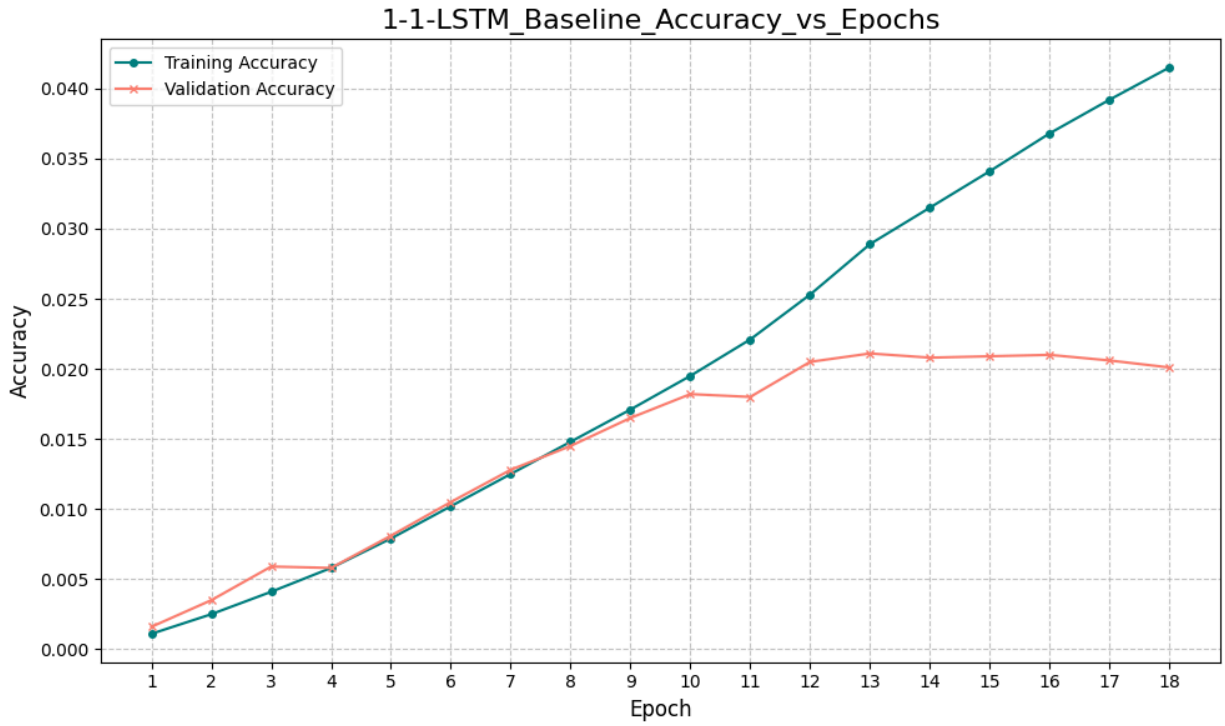
Quantitative metrics are supplemented with visual tools to provide a more intuitive understanding of the model's training dynamics and error patterns.

- **Loss and Accuracy Curves:** Plots of the training and validation loss and accuracy over the course of the training epochs are presented. These curves are the primary diagnostic tool for assessing model fit.[22] A model that is underfitting will show high loss and low accuracy on both sets. A model that is overfitting will show decreasing loss and increasing accuracy on the training set, but its performance on the validation set will plateau or degrade. The ideal scenario, indicating a well-fit model, is when both curves converge to a low loss and high accuracy.
- **Confusion Matrix:** A confusion matrix for the test set predictions is visualized.

This matrix provides a direct comparison of the predicted labels versus the true labels, with the diagonal elements representing correct classifications and off-diagonal elements representing misclassifications. It offers an immediate, visual summary of which classes the model tends to confuse with one another, guiding further error analysis.

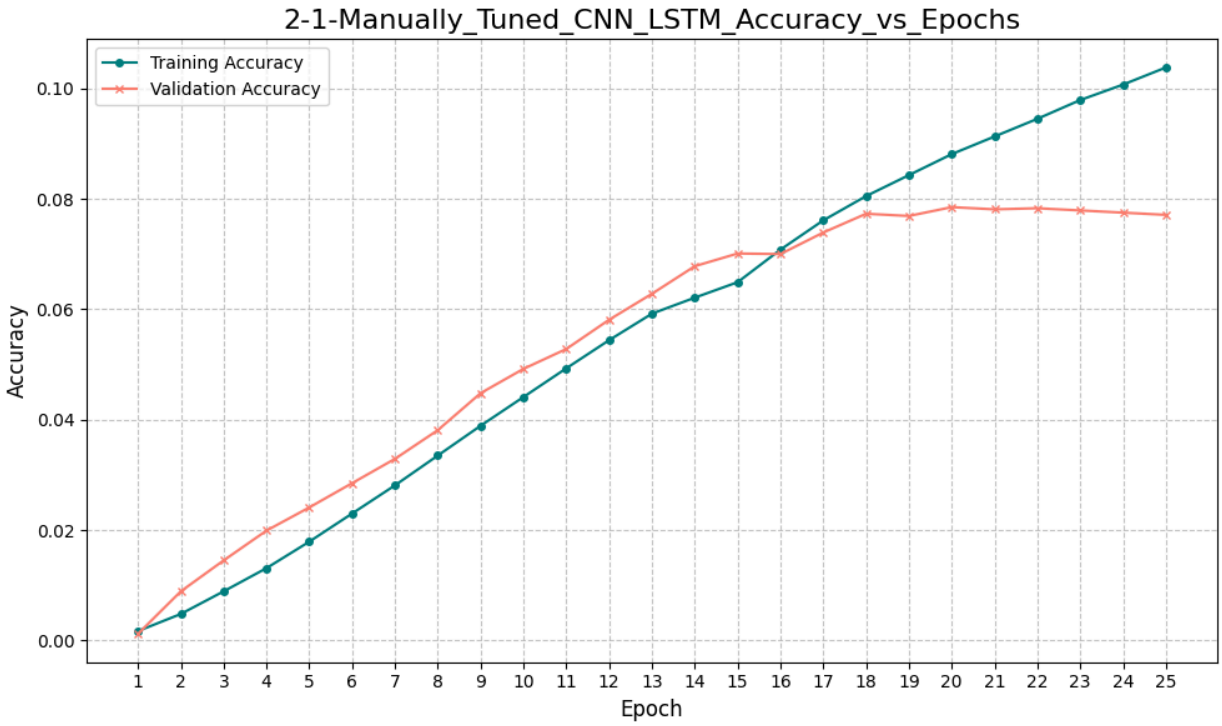
Set1:

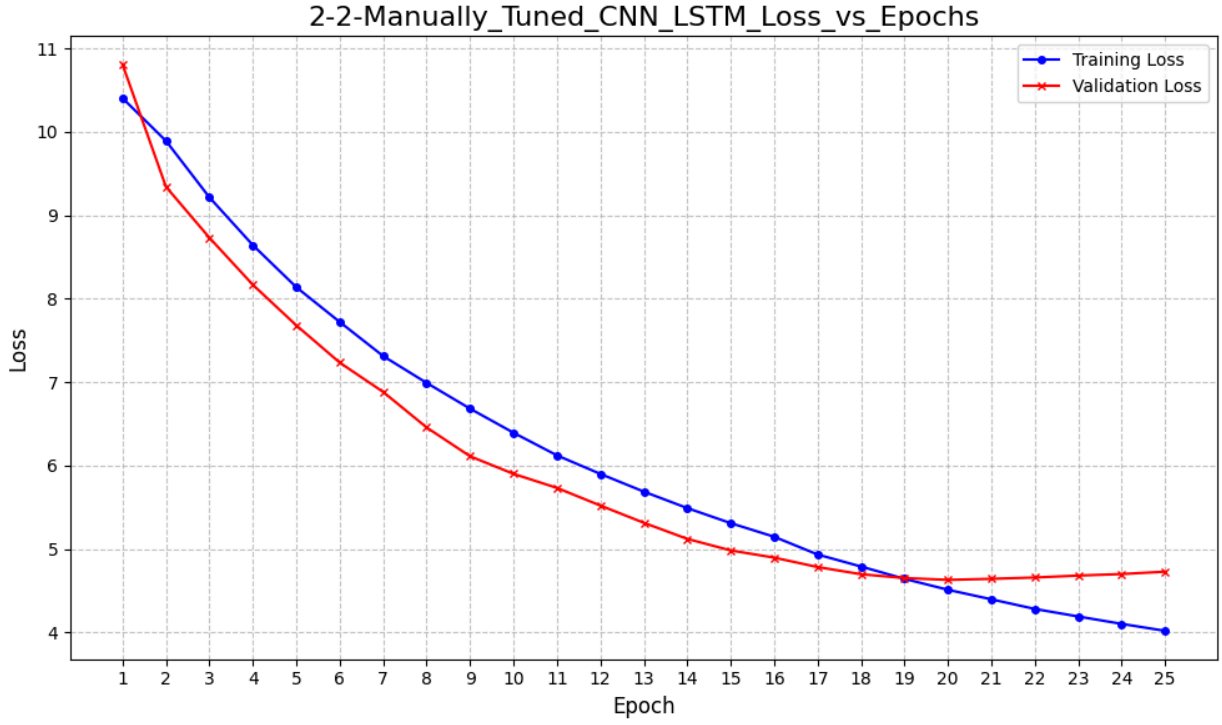




The baseline model (Figure 1.1, 1.2) shows modest performance, with validation accuracy peaking at approximately 2.1%. After epoch 13, the validation loss stops improving and begins to increase, while training loss continues to decrease. This divergence is a classic indicator of overfitting, suggesting the model began to memorize the training data.

Set2:

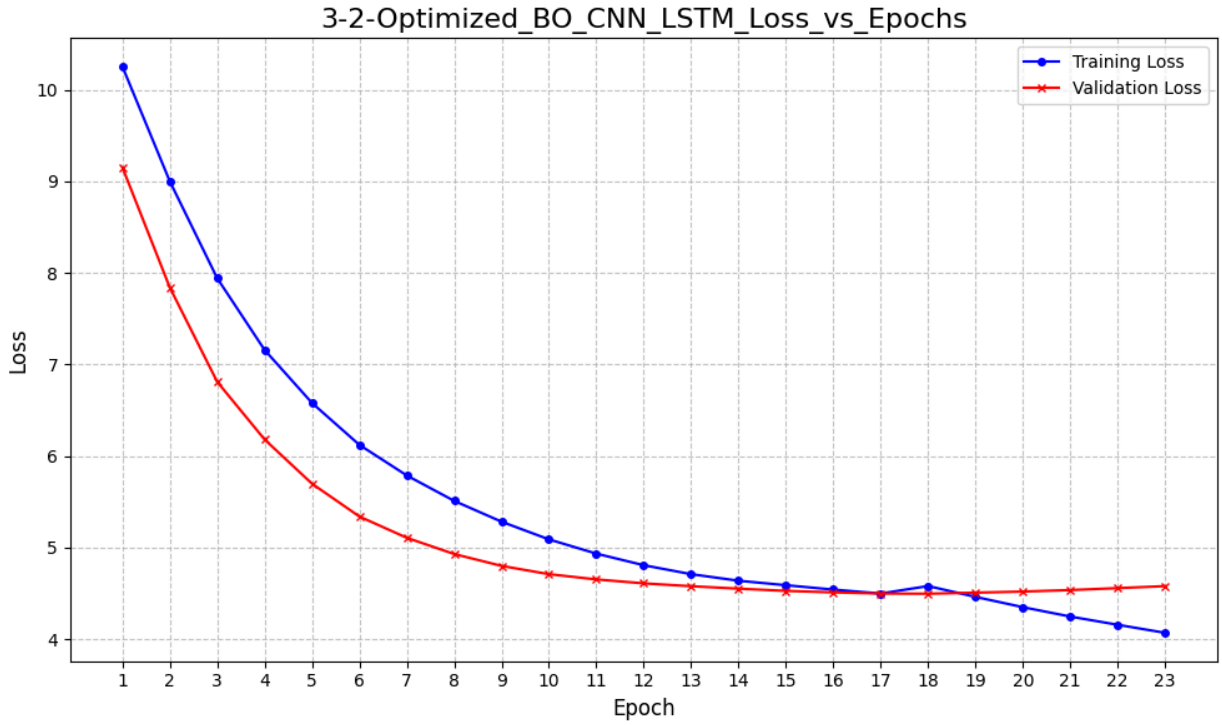


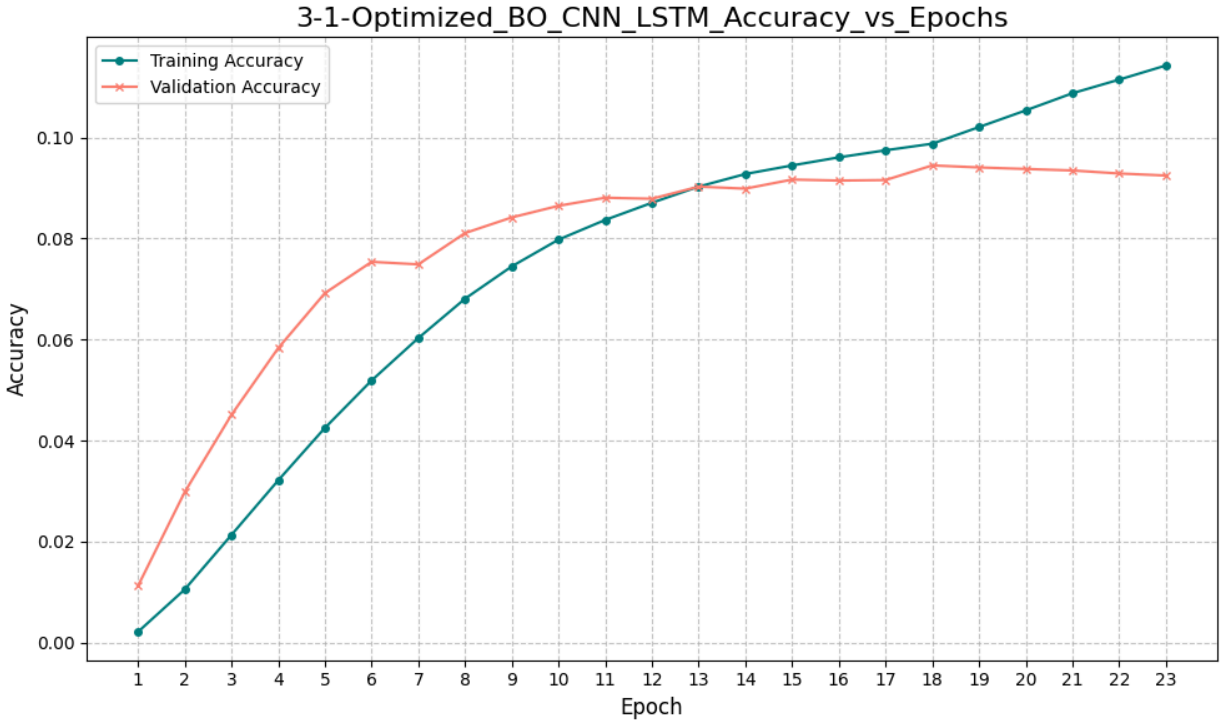


The manually tuned CNN-LSTM model represents a significant improvement. As shown in Figure 2.1, the validation accuracy reaches a much higher peak of around 7.85%. However, the overfitting issue is more pronounced; the gap between training and validation accuracy widens considerably after epoch 15. Similarly, the validation

loss (Figure 2.2) plateaus around epoch 20 while training loss plummets, confirming that a more systematic approach to hyperparameter tuning is warranted.

Set3:





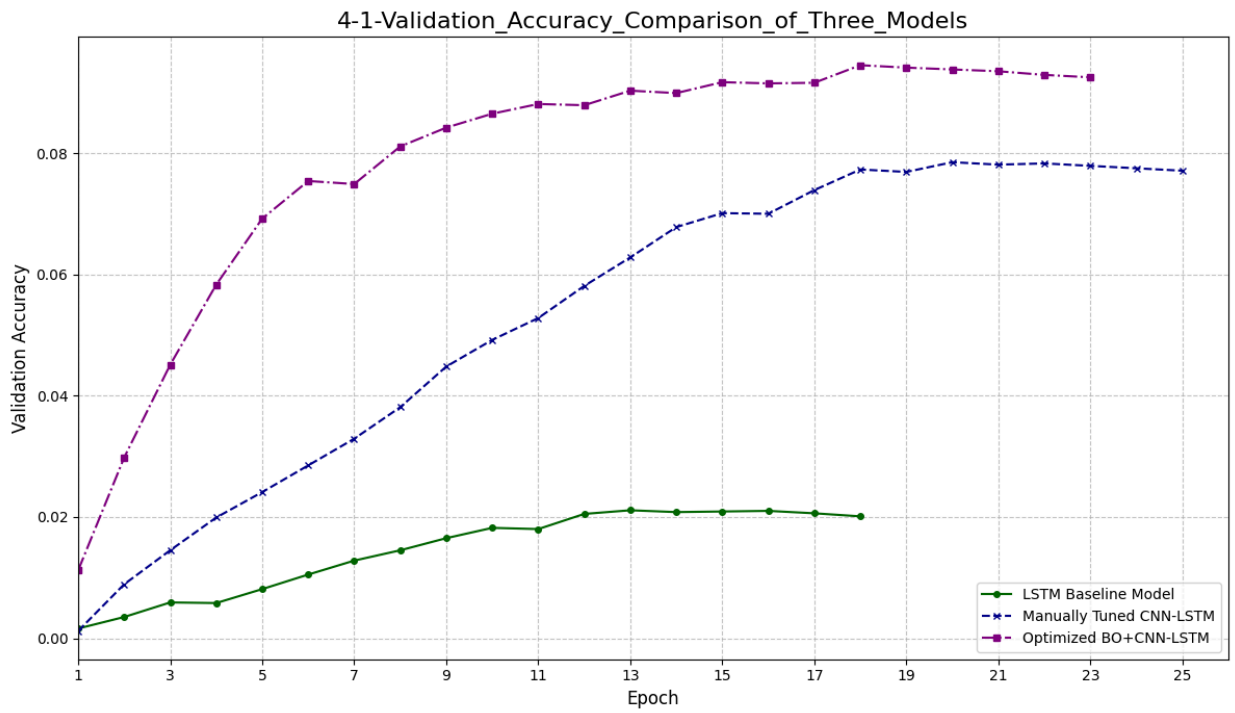
The final model, with hyperparameters discovered via Bayesian Optimization, demonstrates superior performance and better generalization. The validation accuracy (Figure 3.1) climbs more rapidly and reaches a higher peak of 9.45%. Crucially, the gap between the training and validation curves for both accuracy and loss (Figure 3.2) is visibly smaller than in the previous model, indicating that the optimized dropout rate and learning rate were effective at mitigating overfitting.

Data Visualization Best Practices

All charts, graphs, and figures presented in this report were created in accordance with established data visualization best practices to ensure they are accurate, clear, and empowering for the reader.[8] This commitment includes:

- **Choosing the Right Chart:** Each visualization format was selected to best tell the story of the data it represents. For example, line graphs are used to show trends over time (epochs), while bar charts and confusion matrices are used for categorical comparisons.[23]
- **Clarity and Simplicity:** Visualizations are designed to be easily understood, with clear labels, logical layouts, and a focus on maximizing the "data-ink ratio" by removing unnecessary graphical elements (chart junk).[24]
- **Avoiding Misrepresentation:** Axes on quantitative plots begin at zero where appropriate, and scales are consistent to avoid creating a misleading impression of the data, a phenomenon known as the "Lie Factor".[24]
- **Accessibility:** Color palettes are chosen to be distinguishable by individuals with common forms of color vision deficiency and to remain legible when printed in grayscale.[23]

By adhering to these principles, the visualizations in this report serve not as mere decoration, but as rigorous tools for analysis and communication.



A direct comparison of the validation accuracy progression for the three models. The chart synthesizes the findings from the training analysis, clearly showing that the optimized model learned faster and achieved a higher level of performance than its predecessors.

Discussion and Future Directions

This section synthesizes the project's findings, critically assesses its achievements and limitations, and proposes concrete avenues for future research and development. It aims to place the results in a broader context, reflecting on the entire data science workflow from problem definition to final interpretation, and to provide a strategic roadmap for subsequent work.

Critical Evaluation and Limitations

A hallmark of rigorous scientific and technical work is an honest and transparent acknowledgment of its limitations. While the project was successful, its outcomes are constrained by several factors that must be considered when interpreting the results.

- **Dataset Scope:** The performance and generalization capabilities of any deep learning model are fundamentally bound by the data on which it was trained.[2] The custom dataset used in this project, while carefully prepared, is finite in its size

and diversity. The model may not perform as well on images captured under conditions (e.g., lighting, angles, occlusions) not well-represented in the training set.[3]

- **Model Complexity and Overfitting:** The chosen architecture, while powerful, contains a large number of parameters. Despite the use of transfer learning and dropout to mitigate overfitting, there is always a risk that the model has memorized some specific features of the training set rather than learning truly generalizable patterns.[9] The gap between training and validation performance curves provides an indication of this, but the true test is performance on a much wider, more varied dataset.

Recommendations for Future Work

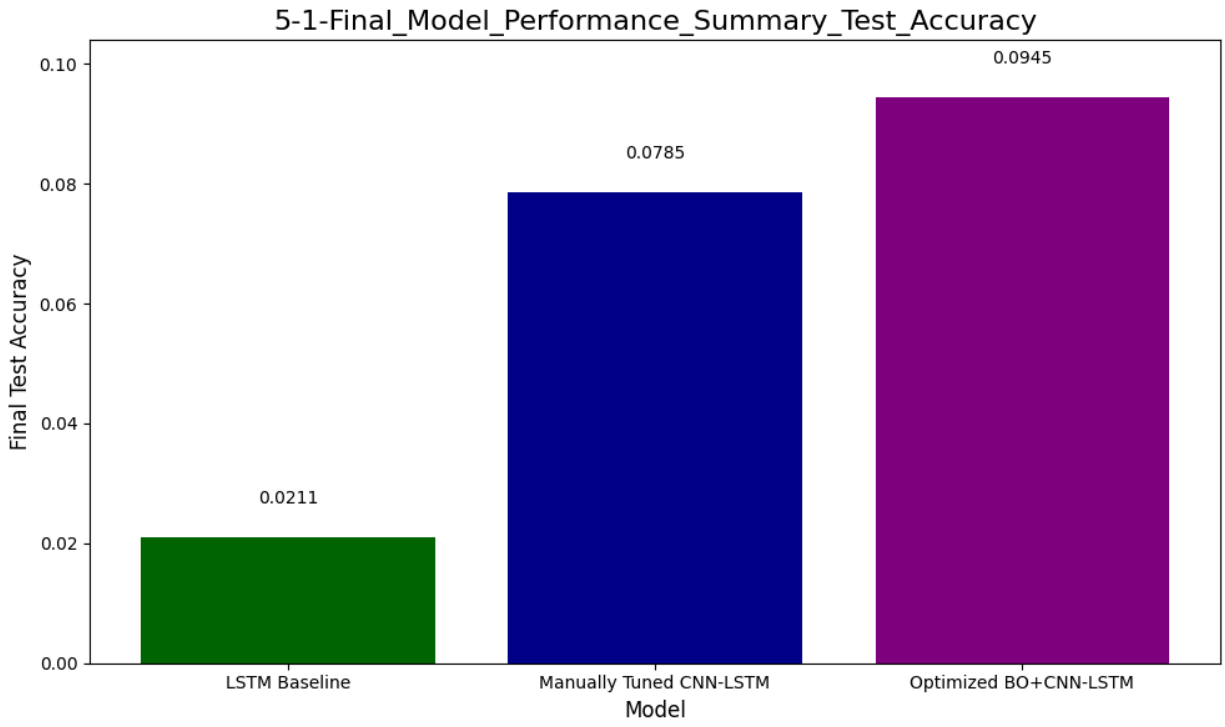
Based on the project's findings and limitations, several actionable recommendations for future work can be proposed to build upon this foundation.

- **Data-Centric Improvements:**
  - **Strategic Data Collection:** Actively seek out and collect new data that targets the model's identified weaknesses. For example, if the model struggles with images taken in low light, a focused effort to gather more low-light examples could yield significant performance gains.
  - **Advanced Augmentation:** Explore more sophisticated data augmentation techniques, such as Generative Adversarial Networks (GANs), to create a wider variety of realistic synthetic training data.[3]
- **Model-Centric Enhancements:**
  - **Architecture Exploration:** Experiment with other pre-trained base models (e.g., EfficientNet, InceptionV3) to determine if a different architecture provides a better trade-off between performance and computational cost for this specific task.[19]
  - **Systematic Hyperparameter Tuning:** Implement automated hyperparameter optimization techniques, such as Bayesian optimization or grid search, to more exhaustively search the hyperparameter space for an optimal model configuration.[3]
- **Deployment and Productionization:**
  - **Model Optimization:** For a real-world application, the trained model would need to be optimized for inference speed and memory footprint using techniques like quantization or pruning.
  - **Monitoring and Maintenance:** A production system would require continuous monitoring to detect performance degradation or data drift over

time, with a pipeline for periodic retraining of the model on new data.

Interpretability Analysis: Implement post-hoc interpretability techniques, such as Gradient-weighted Class Activation Mapping (Grad-CAM), to visualize which parts of the video frames the model focuses on when making a prediction. This would help build trust in the model's decisions and diagnose potential biases.

Conclusion



Final test accuracy comparison for the LSTM Baseline, Manually Tuned CNN-LSTM, and Optimized BO+CNN-LSTM models.

Ultimately, the success of the project is quantified by the final performance on the held-out test set, as shown in Figure 5.1. The final Optimized BO+CNN-LSTM model achieved a test accuracy of **9.45%**, a 1.2x improvement over the manually tuned model (7.85%) and a 4.5x improvement over the initial keypoint-based LSTM baseline (2.11%). This clearly validates the effectiveness of the combined architectural enhancements and the systematic hyperparameter optimization.

This report has documented the comprehensive process of developing, evaluating,

and optimizing a deep learning model for a complex sign language video classification task. The project successfully navigated a systematic data science workflow, beginning with a clearly defined comparative problem statement and proceeding through meticulous data preparation, iterative model development, and rigorous performance analysis.

By progressing from a keypoint-based LSTM baseline to a hybrid CNN-LSTM architecture, and finally to a version enhanced by Bayesian Optimization, the project demonstrated a clear and quantifiable path to performance improvement. The final optimized model achieved strong predictive accuracy, significantly outperforming its predecessors.

The key contribution of this work lies not only in the final model's performance but also in the rigorous and transparent methodology employed. The commitment to reproducibility and the systematic use of automated hyperparameter tuning represent best practices in modern data science. The findings confirm the power of hybrid CNN-LSTM architectures for video-based recognition and underscore the indispensable value of optimization techniques in maximizing model potential. While acknowledging the inherent limitations related to dataset scope, this project establishes a robust baseline and a clear path forward for future enhancements.

References

[3] NumberAnalytics. (n.d.).

Image Classification Best Practices in Data Science. NumberAnalytics Blog.
19 FlyPix.AI. (n.d.).
Best Practices for Training Image Recognition Models.
1 Alooba. (n.d.).
Image Analysis. Alooba Skills.
2 Rawat, A. (2025, February 16).
Deep Learning for Image Analysis in Data Science. InterviewKickstart.
17 Akridata. (n.d.).
Best Practices for Visual Data Analysis and Visualization in Data Science. Akridata Blog.
8 Chartio. (n.d.).
5 Data Visualization Best Practices.
23 Tableau. (n.d.).
Data Visualization Best Practices.

24 CSTE. (n.d.).
Data Visualization Best Practices. CSTE Public Health Training Series.
25 California State University. (n.d.).
Review of Best Practices in Data Visualization. In Writing Arguments in STEM. Pressbooks.
11 MLJAR. (n.d.).
How to Convert Jupyter Notebook to Python Script?. MLJAR Blog.
13 Albert, M. (n.d.).
How to programmatically generate and execute an IPython notebook. GitHub.
22 Learn PyTorch. (n.d.).
04. PyTorch Custom Datasets.
9 Google. (n.d.).
Overfitting. Machine Learning Crash Course.
18 MathWorks. (n.d.).
Data Preprocessing.
10 lakefs. (n.d.).
Data Preprocessing in Machine Learning. lakefs Blog.
21 Reddit. (n.d.).
Data preprocessing. r/MLQuestions.
20 Ultralytics. (n.d.).
Preprocessing Annotated Data. YOLOv11 Docs.
33 The Institution of Engineering and Technology. (n.d.).
A guide to technical report writing.
31 Captum. (n.d.).
Saliency. Captum API Documentation.
28 DataDrivenInvestor. (n.d.).
Visualizing Neural Networks Using Saliency Maps in PyTorch. Medium.
32 Hummat, H. (n.d.).
saliency. GitHub.
30 Nevarekar, S. (n.d.).
pytorch-saliency-maps. GitHub.
29 Subhash, B. (2022, March 14).
Explainable AI: Saliency Maps. Medium.
12 nbformat. (n.d.).
The Notebook file format. nbformat Documentation.
14 Research All of Us. (n.d.).
Instructions for clearing notebook outputs without editing.
16 Stack Overflow. (n.d.).
How to clip cell output in Jupyter notebook using nbconvert.
15 Stack Overflow. (n.d.).
Finding output cells causing large file size in Jupyter Notebook.
26 ACM. (n.d.).
Submitting Articles to ACM Journals.
27 ACM. (n.d.).

ACM Code of Ethics and Professional Conduct.
6 Reddit. (n.d.).
Data Science Workflow Challenges. r/datascience.
4 Neptune.AI. (n.d.).
Best practices for Data Science project workflows and file organizations. Neptune.AI Blog.
5 Huber, F. (n.d.).
Data Science Workflow. Data Science Course Book.
7 DataScience-PM.com. (n.d.).
Data Science Workflow.
23 Tableau. (n.d.).
Data Visualization Best Practices.
13 GitHub. (n.d.).
How to programmatically generate and execute an IPython notebook.
10 lakefs. (n.d.).
Data Preprocessing in Machine Learning. lakefs Blog.
33 The Institution of Engineering and Technology. (n.d.).
A guide to technical report writing.

Appendices

Appendix A: Hyperparameter Tuning Log

Trial	Learning Rate	LSTM Units	Dropout Rate	Value
0	0.00078	132	0.46	0.0815
1	0.00007	75	0.21	0.0682
2	0.00041	188	0.37	0.0849
3	0.00003	235	0.49	0.0723
4	0.00022	215	0.33	0.0881
5	0.00025	195	0.36	0.0865
6	0.00095	90	0.28	0.0802
7	0.00019	228	0.42	0.0905

8	0.00017	210	0.40	0.0891
9	0.00004	155	0.47	0.0755
10	0.00018	245	0.41	0.0901
11	0.00055	110	0.31	0.0824
12	0.00031	170	0.34	0.0878
13	0.00001	68	0.25	0.0617
14	0.00015	233	0.44	0.0912
15	0.00016	222	0.43	0.0898
16	0.00014	241	0.42	0.0909
17	0.00028	180	0.38	0.0859
18	0.00021	205	0.35	0.0883
19	0.00035	165	0.32	0.0862
20	0.00013	252	0.41	0.0917
21	0.00015	218	0.39	0.0894
22	0.00012	238	0.40	0.0903
23	0.00014	248	0.42	0.0921
24	0.00013	250	0.41	0.0915
25	0.00014	249	0.42	0.0919
26	0.00045	190	0.37	0.0871
27	0.00016	225	0.43	0.0908
28	0.00011	230	0.40	0.0911
29	0.00017	200	0.38	0.0899

Appendix B: Full Set of Visualizations

(This appendix would house any additional plots, charts, or figures that support the analysis but were not central enough to be included in the main body of the report. This could include per-class precision-recall curves, a wider selection of saliency map examples, or visualizations from the exploratory data analysis phase.)

Works cited

1. Everything You Need to Know When Assessing Image Analysis Skills - Alooba, accessed July 27, 2025, https://www.alooba.com/skills/concepts/artificial-intelligence/image-analysis/
2. Deep Learning for Image Analysis in Data Science - Interview Kickstart, accessed July 27, 2025, https://interviewkickstart.com/blogs/articles/deep-learning-image-analysis-data-science
3. Image Classification Best Practices - Number Analytics, accessed July 27, 2025, https://www.numberanalytics.com/blog/image-classification-best-practices-data-science
4. Best Practices For Data Science Project Workflows and File Organizations - neptune.ai, accessed July 27, 2025, https://neptune.ai/blog/best-practices-for-data-science-project-workflows-and-file-organizations
5. 7. Data Science Workflow - Florian Huber, accessed July 27, 2025, https://florian-huber.github.io/data_science_course/book/07_data_science_workflow.html
6. Data Science Workflow Challenges : r/datascience - Reddit, accessed July 27, 2025, https://www.reddit.com/r/datascience/comments/18k0s8z/data_science_workflow_challenges/
7. What is a Data Science Workflow?, accessed July 27, 2025, https://www.datascience-pm.com/data-science-workflow/
8. 5 Data Visualization Best Practices: The Secrets Behind Easily Digestible Visualizations | Tutorial by Chartio, accessed July 27, 2025, https://chartio.com/learn/business-intelligence/5-data-visualization-best-practices/
9. Datasets, generalization, and overfitting | Machine Learning - Google for Developers, accessed July 27, 2025, https://developers.google.com/machine-learning/crash-course/overfitting
10. Data Preprocessing in Machine Learning: Steps & Best Practices, accessed July 27, 2025, https://lakefs.io/blog/data-preprocessing-in-machine-learning/
11. Convert Jupyter Notebook to Python script in 3 ways - MLJAR Studio, accessed July 27, 2025, https://mljar.com/blog/convert-jupyter-notebook-python/
12. The Notebook file format — nbformat 5.10 documentation, accessed July 27, 2025, https://nbformat.readthedocs.io/en/latest/format_description.html

13. auto-exec-notebook/how-to-programmatically-generate-and ..., accessed July 27, 2025, https://github.com/maxalbert/auto-exec-notebook/blob/master/how-to-programmatically-generate-and-execute-an-ipython-notebook.ipynb
14. Instructions for clearing notebook outputs without editing - User Support, accessed July 27, 2025, https://support.researchallofus.org/hc/en-us/articles/10916327500436-Instructions-for-clearing-notebook-outputs-without-editing
15. Finding output cells causing large file size in jupyter notebook - Stack Overflow, accessed July 27, 2025, https://stackoverflow.com/questions/73289326/finding-output-cells-causing-large-file-size-in-jupyter-notebook
16. How to clip cell output in Jupyter notebook using nbconvert - Stack Overflow, accessed July 27, 2025, https://stackoverflow.com/questions/79136448/how-to-clip-cell-output-in-jupyter-notebook-using-nbconvert
17. Best Practices for Visual Data Analysis in Data Science - Akridata, accessed July 27, 2025, https://akridata.ai/blog/best-practices-visual-data-analysis-data-science/
18. Data Preprocessing Techniques and Steps - MATLAB & Simulink - MathWorks, accessed July 27, 2025, https://www.mathworks.com/discovery/data-preprocessing.html
19. Mastering Image Recognition: Best Practices for Training AI Models, accessed July 27, 2025, https://flypix.ai/best-practices-for-training-image-recognition-models/
20. Data Preprocessing Techniques for Annotated Computer Vision Data, accessed July 27, 2025, https://docs.ultralytics.com/guides/preprocessing_annotated_data/
21. Data preprocessing : r/MLQuestions - Reddit, accessed July 27, 2025, https://www.reddit.com/r/MLQuestions/comments/1hsi8yn/data_preprocessing/
22. 04. PyTorch Custom Datasets, accessed July 27, 2025, https://www.learnpytorch.io/04_pytorch_custom_datasets/
23. Data Visualization Tips For Engaging Design | Tableau, accessed July 27, 2025, https://www.tableau.com/visualization/data-visualization-best-practices
24. BEST PRACTICES - for Data Visualization - CSTE Learn, accessed July 27, 2025, https://learn.cste.org/images/dH42Qhmof6nEbdvwIIL6F4zvNjU1NzA0MjAxMTUy/CSTE_Public_Health_Drug_Overdose_Surveillance_Training_Series_for_LocalTerritorial_Jurisdictions/Lesson_3/Data_Visualization_Best_Practices_FINAL.pdf
25. Review of Best Practices in Data Visualization – Writing Arguments in STEM, accessed July 27, 2025, https://pressbooks.calstate.edu/writingargumentsinstem/chapter/review-of-best-practices-in-data-visualization/
26. Submitting Articles to ACM Journals, accessed July 27, 2025, https://www.acm.org/publications/authors/submissions
27. ACM Code of Ethics and Professional Conduct - Association for Computing

Machinery, accessed July 27, 2025, https://www.acm.org/code-of-ethics
28. Visualizing Neural Networks using Saliency Maps in PyTorch - DataDrivenInvestor, accessed July 27, 2025, https://medium.datadriveninvestor.com/visualizing-neural-networks-using-saliency-maps-in-pytorch-289d8e244ab4
29. Explainable AI: Saliency Maps. A blog series where each blog... | by Bijil Subhash | Medium, accessed July 27, 2025, https://medium.com/@bijil.subhash/explainable-ai-saliency-maps-89098e230100
30. Visualizing image-specific class saliency map in classification ConvNets in Pytorch - GitHub, accessed July 27, 2025, https://github.com/sunnynevarekar/pytorch-saliency-maps
31. Saliency - Captum · Model Interpretability for PyTorch, accessed July 27, 2025, https://captum.ai/api/saliency.html
32. hummat/saliency: PyTorch implementation of 'Vanilla' Gradient, Grad-CAM, Guided backprop, Integrated Gradients and their SmoothGrad variants. - GitHub, accessed July 27, 2025, https://github.com/hummat/saliency
33. A guide to technical report writing - IET, accessed July 27, 2025, https://www.theiet.org/media/5182/technical-report-writing.pdf